

Aim:

To develop applications for managing **product and task data** using Node.js, Express.js, and MongoDB, and to build a **Task Tracker Dashboard** with a frontend developed using React that allows users to create, search, update, and delete tasks through REST API services.

Description:

This experiment demonstrates how to store and manage product and task data using Node.js, Express, and MongoDB. A backend application is used to insert and retrieve product information from the database. In addition, a Task Tracker Dashboard is developed using React as the frontend and Node.js with Express as the backend to perform CRUD operations on tasks stored in MongoDB.

Experiment 1: Product Storage using MongoDB and Express

To develop a simple **Node.js application** that performs **CRUD (Create, Read, Update, Delete)** operations for product data using MongoDB as the database and Express.js as the server framework.

Concepts Used:

- Node.js – JavaScript runtime used to run the server-side application.
- Express.js – Web framework used to create REST APIs and handle HTTP requests.
- MongoDB – NoSQL database used to store product data in collections.
- MongoDB **Node Driver** – Used to connect the Node.js application to MongoDB.
- **REST API** – Used to perform CRUD operations using HTTP methods (GET, POST, PUT, DELETE).
- **JSON Data Format** – Used to send and receive product data between client and server.

Code:

Server.js:

```
const express = require("express");
const { MongoClient } = require("mongodb");
const app = express();
app.use(express.json());
const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);

let db;

async function connectDB(){
  await client.connect();
  db = client.db("productDB");
  console.log("Mongo Connected Successfully");
}
connectDB();
app.post("/addProduct", async (req,res) => {
  const product = req.body;
  await db.collection("products").insertOne(product);
  res.send("Product Inserted Successfully");
});
app.put("/updateProduct/:name", async (req, res) => {
```

```

const name = req.params.name;
const newData = req.body;

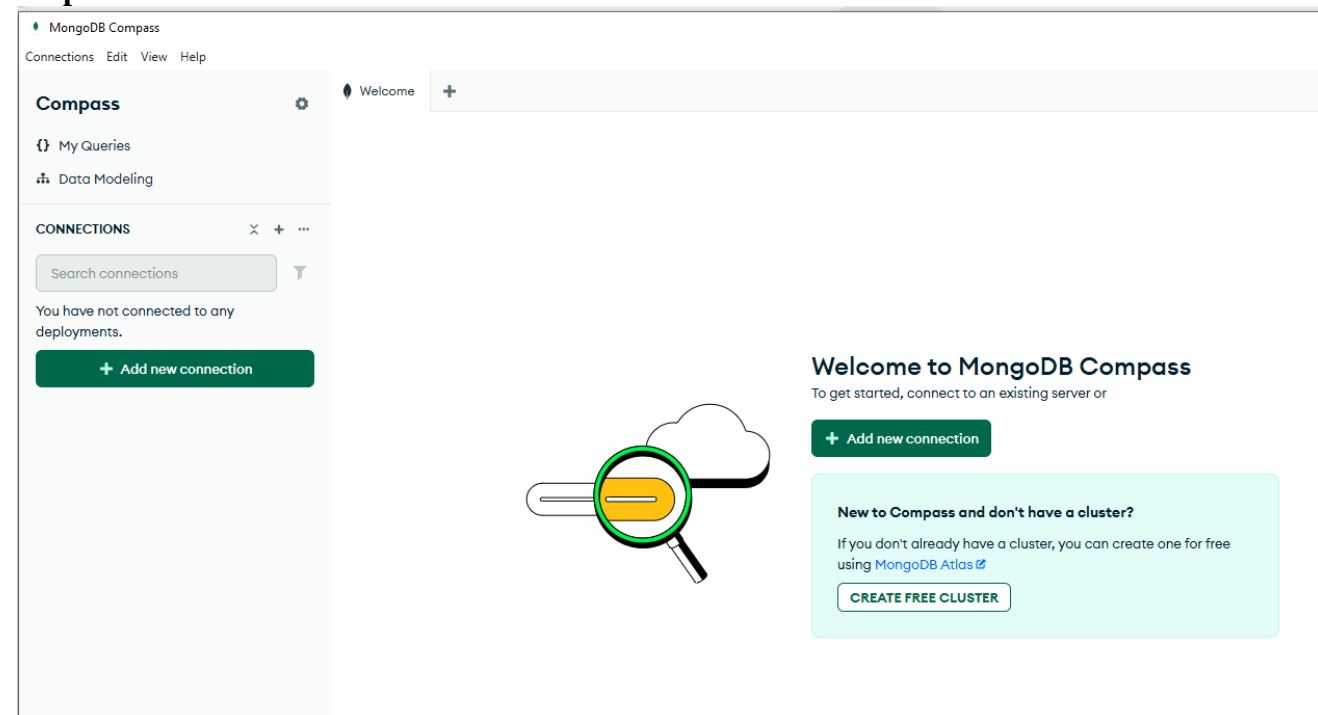
const result = await db.collection("products").updateOne(
  { name: name },
  { $set: newData }
);
res.send("Product Updated Successfully");
});
app.delete("/deleteProduct/:name", async (req, res) => {
  const name = req.params.name;

  const result = await db.collection("products").deleteOne({ name: name });

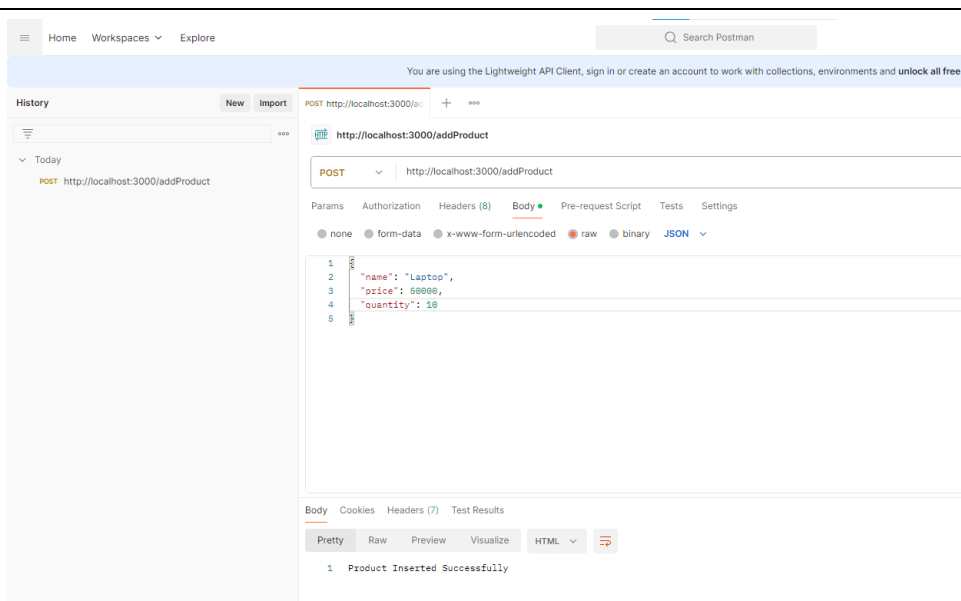
  res.send("Product Deleted Successfully");
});
app.get("/product", async (req, res) => {
  const products = await db.collection("products").find().toArray();
  res.json(products);
});
app.listen(3000, () => {
  console.log("Server running Successfully");
});

```

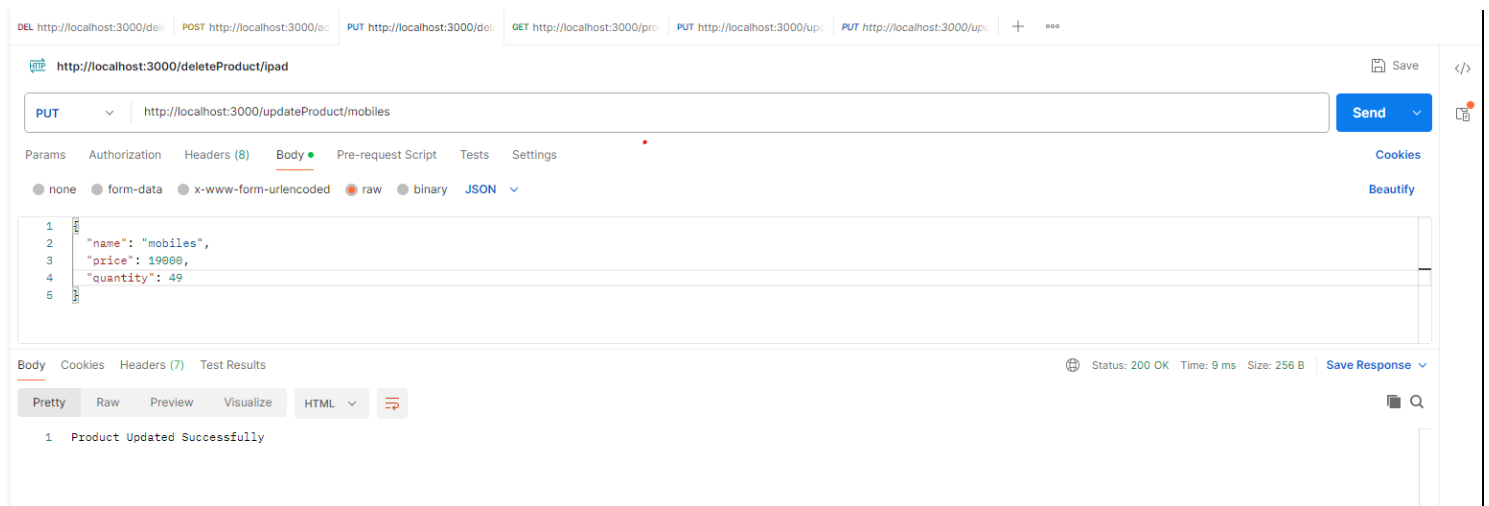
Output:



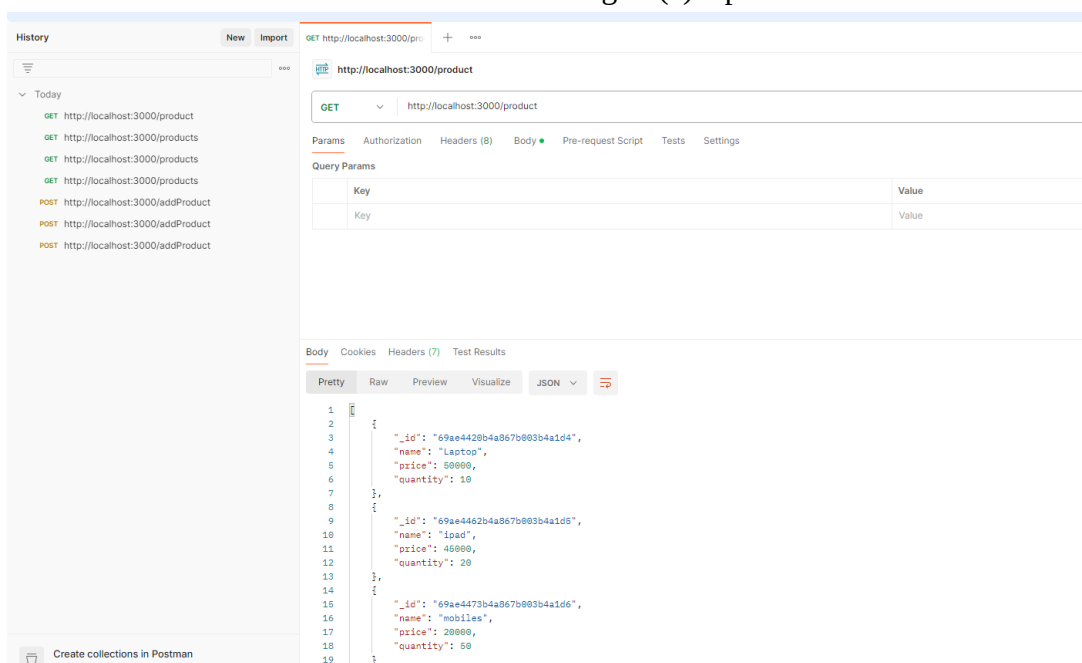
Figure(a) Installation of MongoDB



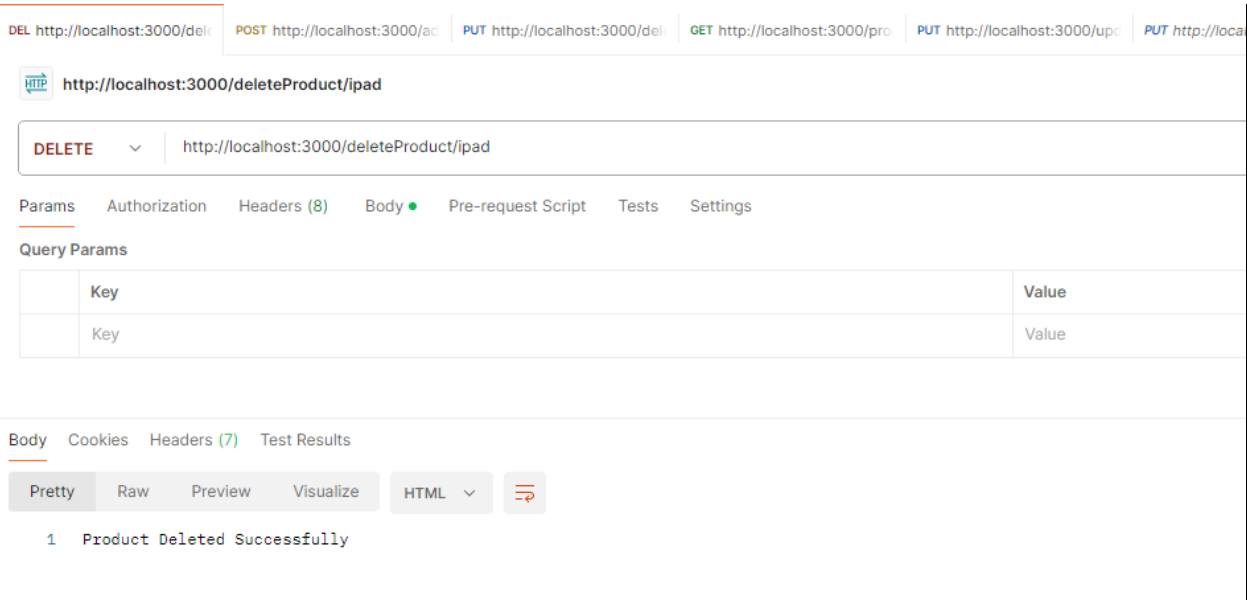
Figure(b) Adding Products



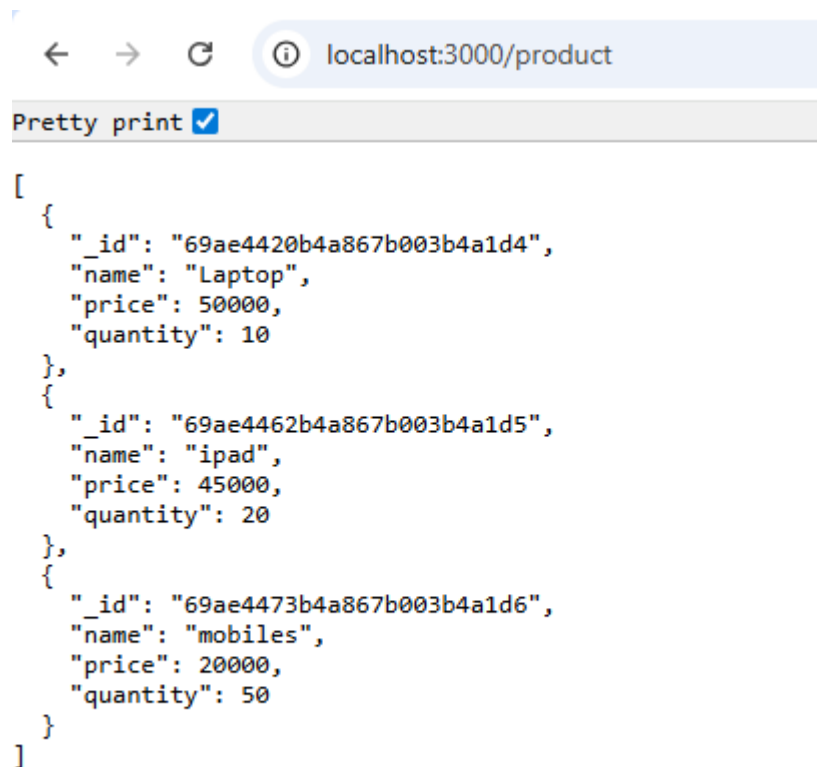
Figure(c) Update Products



Figure(d) selecting Products



Figure(e) Deleting Products



Figure(f) Brower View

Experiment 2: Task Tracker Dashboard using React, Node.js, Express and MongoDB

To develop a **Task Tracker Dashboard** that allows users to create, search, update, and delete tasks using Node.js, Express.js, MongoDB, and React & Backend

Concepts Used:

- React – Used to build the interactive user interface for the task dashboard.
- Node.js – JavaScript runtime used to run the backend server.
- Express.js – Web framework used to create REST API endpoints.
- MongoDB – NoSQL database used to store task data.
- REST API – Used for communication between the frontend and backend.
- CRUD Operations – Create, Read, Update, and Delete tasks.
- JSON Format – Used to exchange data between client and server.

Code:

App.jsx:

```
import { useState, useEffect } from "react";
import "./App.css";
```

```
function App() {
```

```
  const [title, setTitle] = useState("");
  const [tasks, setTasks] = useState([]);
  const [search, setSearch] = useState("");
```

```
  const loadTasks = () => {
    fetch("http://localhost:3001/tasks")
      .then(res => res.json())
      .then(data => setTasks(data));
  };
```

```
  useEffect(() => {
    loadTasks();
  }, []);
```

```
  const addTask = () => {
    if (!title) return;
```

```
    fetch("http://localhost:3001/tasks", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
```

```

    },
    body: JSON.stringify({ title })
  }).then(() => {
    loadTasks();
    setTitle("");
  });
};

const deleteTask = (title) => {
  fetch(`http://localhost:3001/tasks/${title}`, {
    method: "DELETE"
  }).then(loadTasks);
};

const updateTask = (oldTitle) => {
  const newTitle = prompt("Enter new task");
  if (!newTitle) return;
  fetch(`http://localhost:3001/tasks/${oldTitle}`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ title: newTitle })
  }).then(loadTasks);
};

const filteredTasks = tasks.filter(t =>
  t.title && t.title.toLowerCase().includes(search.toLowerCase())
);

return (
  <div className="container">
    <h2>Task Tracker</h2>
    <div className="form-row">
      <input
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        placeholder="Enter Task"
      />
      <button onClick={addTask}>Add</button>
    </div>
    <input
      className="search"
      placeholder="Search task..."
      value={search}
      onChange={(e) => setSearch(e.target.value)}
    />
    <ul>
      {filteredTasks.map((t, i) => (
        <li key={i}>

```

```

      {t.title}
    <div>
      <button onClick={() => updateTask(t.title)}>Edit</button>
      <button onClick={() => deleteTask(t.title)}>Delete</button>
    </div>
  </li>
  )}
</ul>
</div>
);
}

```

export default App;

server.js:

```

const express = require("express");
const cors = require("cors");
const { MongoClient } = require("mongodb");
const app = express();
app.use(cors());
app.use(express.json());
const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);
let db;
async function connectDB() {
  await client.connect();
  db = client.db("taskDB");
  console.log("MongoDB Connected");
}
connectDB();
app.post("/tasks", async (req, res) => {
  const task = req.body;
  await db.collection("tasks").insertOne(task);
  res.send("Task Added");
});
app.get("/tasks", async (req, res) => {
  const tasks = await db.collection("tasks").find().toArray();
  res.json(tasks);
});
app.put("/tasks/:title", async (req, res) => {
  const title = req.params.title;
  const newData = req.body;
  await db.collection("tasks").updateOne(
    { title: title },
    { $set: newData }
  );
  res.send("Task Updated");
}

```

```
});  
app.delete("/tasks/:title", async (req, res) => {  
  const title = req.params.title;  
  await db.collection("tasks").deleteOne({ title: title });  
  res.send("Task Deleted");  
});  
app.listen(3001, () => {  
  console.log("Server Running on Port 3001");  
});
```

App.css:

```
body {  
  font-family: Arial, sans-serif;  
  background: #eef2f7;  
  margin: 0;  
  height: 100vh;  
  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
.container {  
  width: 420px;  
  background: white;  
  padding: 25px;  
  border-radius: 8px;  
  box-shadow: 0 3px 10px rgba(0,0,0,0.1);  
  text-align: center;  
}  
  
h2 {  
  margin-bottom: 20px;  
  color: #333;  
}  
  
.form-row {  
  display: flex;  
  gap: 8px;  
  justify-content: center;  
}  
  
input {  
  flex: 1;  
  padding: 8px;  
  border: 1px solid #ccc;  
  border-radius: 4px;
```



```

button {
  padding: 8px 14px;
  border: none;
  background: #4CAF50;
  color: white;
  border-radius: 4px;
  cursor: pointer;
}
button:hover {
  background: #43a047;
}
ul {
  list-style: none;
  padding: 0;
  margin-top: 20px;
}
li {
  background: #f7f7f7;
  margin: 8px 0;
  padding: 8px 12px;
  border-radius: 4px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
li button {
  background: #e53935;
}
li button:hover {
  background: #c62828;
}

```

Output:

Task Tracker

study	Edit Delete
playing	Edit Delete
sleep	Edit Delete
gym	Edit Delete

Figure(g) Add Task

The image shows a web application titled "Task Tracker". At the top, there is a text input field labeled "Enter Task" and a green "Add" button. Below the input field is a search bar labeled "Search task...". The main area of the application displays a list of tasks: "study", "playing", "code", and "sleep". Each task is in a light gray box and has two red buttons to its right: "Edit" and "Delete".

Figure(h): Selection of tasks

The image shows the "Task Tracker" interface with a modal dialog box open. The modal is titled "localhost:5173 says" and contains the text "Enter new task". Below this text is a text input field with the word "studing" entered. At the bottom of the modal are two buttons: "OK" (blue) and "Cancel" (light blue). The background of the application is dimmed, showing the same "Task Tracker" interface as in Figure(g), but with an additional task "gym" at the bottom of the list.

Figure(i): Updating of tasks

Task Tracker

study

EditDelete

playing

EditDelete

sleep

EditDelete

Figure(j): Deletion on Tasks

Task Tracker

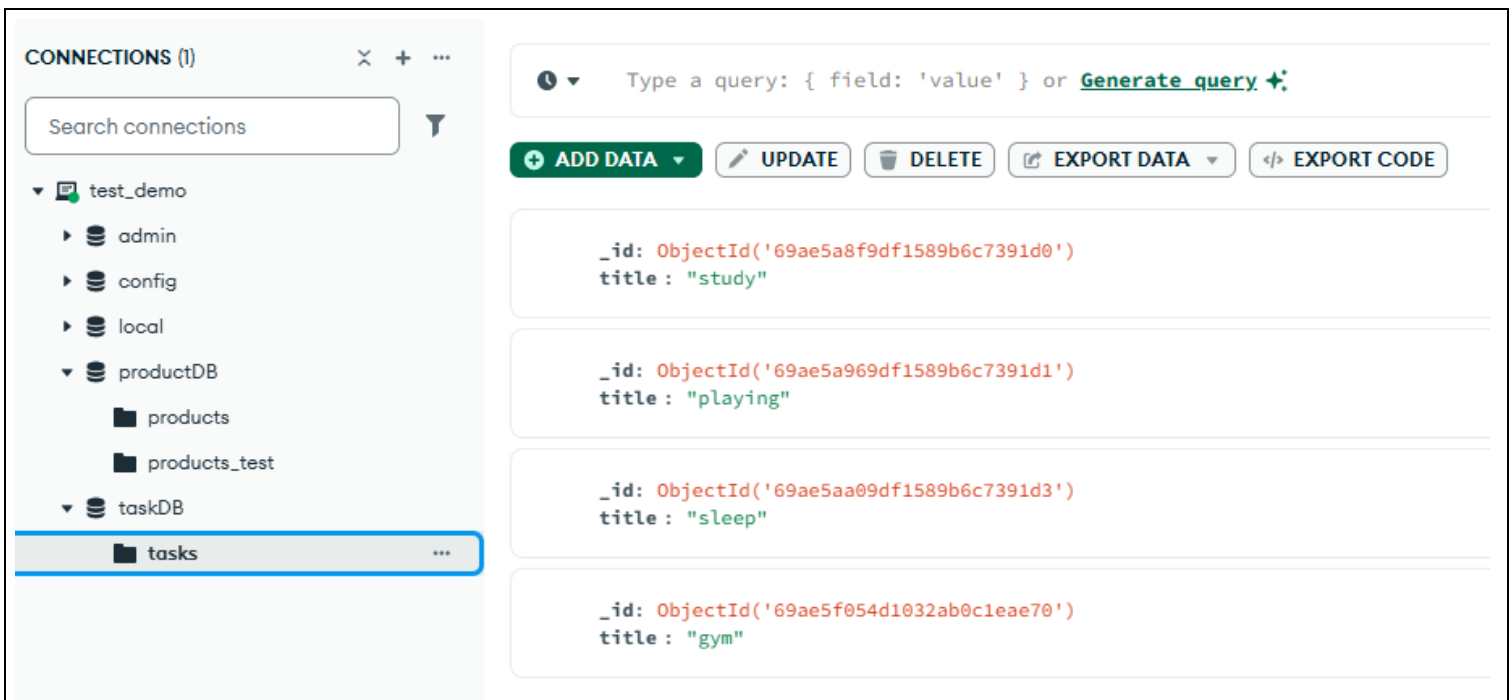
study

EditDelete

sleep

EditDelete

Figure(k): Searching on Tasks



Figure(1): MongoDB view

Result:

Thus, the applications were successfully developed to manage **product and task data** using Node.js, Express.js, and MongoDB. The product storage system allowed inserting and retrieving product information, and the Task Tracker Dashboard built using React enabled users to add, search, update, and delete tasks successfully.